Cuaderno de estudio 06

Función Transferencia (Python)

Función de transferencia, análisis temporal y análisis espectral

Carrera: Ingeniería Biomédica
Herramientas de simulación: Python, NumPy, SciPy y Matplotlib

---

1. Introducción

En las clases anteriores se estudiaron las ecuaciones diferenciales y las variables de estado como herramientas para describir sistemas dinámicos.

En este capítulo se introduce una herramienta especialmente útil para el análisis de sistemas lineales e invariantes en el tiempo (LTI): la **función de transferencia**.

El objetivo no es volver a desarrollar las ecuaciones diferenciales ni las variables de estado, sino utilizarlas como punto de partida para estudiar:

- función de transferencia;
- polos, ceros y ganancia;
- respuesta temporal;
- respuesta al impulso y al escalón;
- parámetros de sistemas de primer y segundo orden;
- respuesta en frecuencia;
- magnitud y fase;
- frecuencia de corte y ancho de banda;
- diagramas de Bode;
- resonancia;
- relación entre comportamiento temporal y espectral;
- simulación mediante Python;
- aplicaciones a sistemas de interés biomédico.

> **Idea central:** una misma función de transferencia permite estudiar cómo responde un sistema tanto en el dominio del tiempo como en el dominio de la frecuencia.

---

2. Objetivos de aprendizaje

Al finalizar el capítulo, el estudiante debería ser capaz de:

1. Interpretar una función de transferencia.
2. Identificar polos, ceros y ganancia.
3. Obtener y analizar la respuesta temporal de un sistema.
4. Interpretar la respuesta al impulso y al escalón.
5. Reconocer los parámetros característicos de sistemas de primer y segundo orden.
6. Analizar la respuesta en frecuencia.
7. Determinar frecuencia de corte y ancho de banda.
8. Interpretar magnitud y fase.
9. Construir e interpretar diagramas de Bode.
10. Relacionar polos y ceros con el comportamiento del sistema.
11. Simular sistemas dinámicos mediante Python.
12. Analizar sistemas de interés para Ingeniería Biomédica.

---

3. Del modelo matemático a la función de transferencia

Para un sistema lineal e invariante en el tiempo, suponiendo condiciones iniciales nulas:

$$H(s) = \frac{Y(s)}{X(s)}$$

donde:

- $(X(s)$: transformada de Laplace de la entrada;
- $ Y(s)$: transformada de Laplace de la salida;
- $ H(s)$: función de transferencia;
- $ s$: variable compleja de Laplace.

La función de transferencia describe una propiedad del **sistema**, no una señal particular.

Una vez conocida $ H(s)$, se puede estudiar la respuesta ante diferentes entradas.

---

4. ¿Por qué es útil la función de transferencia?

Permite estudiar de manera sistemática:

- estabilidad;

- rapidez de respuesta;
- error estacionario;
- sobreimpulso;
- frecuencia de corte;
- ancho de banda;
- resonancia;
- atenuación;
- amplificación;
- desfase.

También permite comparar sistemas sin resolver nuevamente las ecuaciones diferenciales para cada señal de entrada.

---

5. Polos, ceros y ganancia

Una función de transferencia puede escribirse como:

$$H(s) = K\frac{(s - z_1)(s - z_2)\dots}{(s - p_1)(s - p_2)\dots}$$

donde:

- K : ganancia;
- z_i : ceros;
- p_i : polos.

Los polos son especialmente importantes porque determinan gran parte de la dinámica y la estabilidad.

Ejemplo

$$H(s) = \frac{5}{s + 2}$$

posee un polo en:

$$s = -2$$

---

6. Análisis temporal

El análisis temporal estudia cómo evoluciona la salida en función del tiempo.

Se pueden utilizar entradas como:

- impulso;
- escalón;
- rampa;
- exponencial;
- sinusoidal;
- señal arbitraria.

Dos respuestas fundamentales son la respuesta al impulso y la respuesta al escalón.

---

7. Respuesta al impulso

Para el impulso de Dirac:

$$\mathcal{L}\{\delta(t)\} = 1$$

por lo tanto:

$$X(s) = 1$$

y:

$$Y(s) = H(s)$$

La respuesta temporal es:

$$h(t) = \mathcal{L}^{-1}\{H(s)\}$$

Para sistemas LTI, la respuesta impulsional permite determinar la respuesta frente a una entrada arbitraria mediante la convolución.

---

8. Respuesta al escalón

Para el escalón unitario:

$$\mathcal{L}\{u(t)\} = \frac{1}{s}$$

por lo tanto:

$$X(s) = \frac{1}{s}$$

y:

$$Y(s) = \frac{H(s)}{s}$$

La respuesta al escalón es una de las herramientas más utilizadas para caracterizar sistemas dinámicos.

---

9. Parámetros característicos de la respuesta temporal

En una respuesta al escalón pueden estudiarse:

Tiempo de subida

Tiempo necesario para pasar desde un porcentaje bajo hasta uno alto del valor final, por ejemplo 10–90 %.

Tiempo de establecimiento

Tiempo necesario para que la respuesta permanezca dentro de una banda alrededor del valor final, por ejemplo ±2 % o ±5 %.

Sobreimpulso

Diferencia entre el máximo de la respuesta y su valor final, expresada normalmente como porcentaje.

Tiempo de retardo

Tiempo requerido para alcanzar determinado porcentaje del valor final.

Error estacionario

Diferencia entre el valor deseado y el valor alcanzado cuando ha transcurrido suficiente tiempo.

---

10. Sistemas de primer orden

Una forma típica es:

$$G(s) = \frac{K}{\tau s + 1}$$

donde:

- K : ganancia estática;
- τ : constante de tiempo.

El polo es:

$$s = -\frac{1}{\tau}$$

La constante de tiempo determina la rapidez de respuesta.

---

11. Respuesta al escalón de un sistema de primer orden

Para una entrada escalón de amplitud A :

$$y(t) = KA\left(1 - e^{-t/\tau}\right)$$

Valores importantes:

Tiempo	Porcentaje del valor final
τ	63,2 %
2τ	86,5 %
3τ	95,0 %
4τ	98,2 %
$ \tau$	99,3 %

Como aproximación habitual:

$$t_s \approx 4\tau$$

para una banda de establecimiento del 2 %.

---

12. Ejemplo biomédico: sensor de temperatura

Supongamos:

$$G(s) = \frac{1}{10s + 1}$$

La constante de tiempo es:

$$\tau = 10\ s$$

Para un cambio de temperatura de 2 °C:

$$y(t) = 2\left(1 - e^{-t/10}\right)$$

El valor final es 2 °C y aproximadamente a los 40 s se alcanza el 98 % del valor final.

Este modelo representa de forma simplificada la dinámica de un sensor de temperatura.

---

13. Simulación Python: sistema de primer orden

Se utilizarán:

- **NumPy** para cálculo numérico;
- **SciPy** para sistemas dinámicos;
- **Matplotlib** para gráficos.

Instalación:

```shell
pip install numpy scipy matplotlib
```

Respuesta al escalón

```python
import matplotlib.pyplot as plt
from scipy import signal

K = 1
tau = 10

# G(s) = K / (tau*s + 1)
sistema = signal.TransferFunction([K], [tau, 1])

t, y = signal.step(sistema)

plt.figure()
plt.plot(t, y)
plt.grid(True)
plt.title("Respuesta al escalón - Sistema de primer orden")
plt.xlabel("Tiempo [s]")
plt.ylabel("Salida")
plt.show()
```

---

14. Respuesta a una entrada escalón de amplitud arbitraria

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

K = 1
tau = 10

sistema = signal.TransferFunction([K], [tau, 1])

t = np.linspace(0, 60, 1000)

# Escalón de amplitud 2
x = 2 * np.ones_like(t)

t, y, _ = signal.lsim(sistema, U=x, T=t)

plt.figure()
plt.plot(t, x, "--", label="Entrada")
plt.plot(t, y, label="Salida")
plt.grid(True)
plt.xlabel("Tiempo [s]")
plt.ylabel("Amplitud")
plt.title("Sistema de primer orden")
plt.legend()
plt.show()
```

---

15. Respuesta en frecuencia

Para estudiar la respuesta frente a señales sinusoidales se reemplaza:

$$s = j\omega$$

donde:

$$\omega = 2\pi f$$

Entonces:

$$H(j\omega)$$

representa la respuesta en frecuencia.

Puede expresarse mediante:

- magnitud;
- fase.

---

16. Magnitud y fase

$$H(j\omega) = |H(j\omega)|e^{j\phi(\omega)}$$

donde:

- $|H(j\omega)|$: magnitud;
- $\phi(\omega)$: fase.

La magnitud indica cuánto se amplifica o atenúa una señal sinusoidal.

La fase indica el desfase introducido por el sistema.

---

17. Sistema de primer orden en frecuencia

Para:

$$G(s) = \frac{K}{\tau s + 1}$$

se obtiene:

$$G(j\omega) = \frac{K}{1 + j\omega\tau}$$

La magnitud:

$$|G(j\omega)| = \frac{K}{\sqrt{1 + (\omega\tau)^2}}$$

La fase:

$$\phi(\omega) = -\tan^{-1}(\omega\tau)$$

---

18. Frecuencia de corte

Para un sistema de primer orden:

$$\omega_c = \frac{1}{\tau}$$

y:

$$f_c = \frac{1}{2\pi\tau}$$

Para $\tau = 10\ s$:

$$f_c \approx 0,0159\ Hz$$

Un sistema con una constante de tiempo grande responde lentamente y atenúa fuertemente las variaciones rápidas.

---

19. Ejemplo biomédico: filtrado de una señal fisiológica

Consideremos:

$$G(s) = \frac{1}{0,05s + 1}$$

Su frecuencia de corte es:

$$f_c = \frac{1}{2\pi(0,05)} \approx 3,18\ Hz$$

Las componentes de frecuencia significativamente superiores a la frecuencia de corte serán atenuadas.

Este modelo puede utilizarse como aproximación de un filtro pasabajos en una cadena de adquisición de señales fisiológicas.

---

20. Respuesta en frecuencia con Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

K = 1
tau = 0.05

sistema = signal.TransferFunction([K], [tau, 1])

w = np.logspace(-1, 3, 1000)

w, H = signal.freqresp(sistema, w=w)

magnitud = np.abs(H)
fase = np.angle(H, deg=True)

plt.figure()
plt.semilogx(w, magnitud)
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Magnitud")
plt.title("Respuesta en frecuencia")
plt.show()

plt.figure()
plt.semilogx(w, fase)
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Fase [grados]")
plt.title("Fase")
plt.show()
```

---

21. Diagramas de Bode

Los diagramas de Bode representan:

1. magnitud en función de la frecuencia;
2. fase en función de la frecuencia.

La magnitud se expresa normalmente en decibeles:

$$|H(j\omega)|_{dB} = 20 \log_{10} |H(j\omega)|$$

La frecuencia se representa en escala logarítmica.

---

22. Diagrama de Bode utilizando Python

SciPy permite obtener directamente magnitud y fase:

```python
import matplotlib.pyplot as plt
from scipy import signal

K = 1
tau = 0.05

sistema = signal.TransferFunction([K], [tau, 1])

w, mag, phase = signal.bode(sistema)

plt.figure()
plt.semilogx(w, mag)
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Magnitud [dB]")
plt.title("Diagrama de Bode - Magnitud")
plt.show()

plt.figure()
plt.semilogx(w, phase)
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Fase [grados]")
plt.title("Diagrama de Bode - Fase")
plt.show()
```

---

23. Sistemas de segundo orden

Una forma canónica es:

$$G(s) = \frac{K\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

donde:

- K : ganancia;
- ω_n : frecuencia natural;
- ζ : coeficiente de amortiguamiento.

---

24. Clasificación según el amortiguamiento

Condición	Tipo
$\zeta > 1$	Sobreamortiguado
$\zeta = 1$	Críticamente amortiguado
$0 < \zeta < 1$	Subamortiguado
$\zeta = 0$	Sin amortiguamiento

Los sistemas subamortiguados pueden presentar oscilaciones y sobreimpulso.

---

25. Ejemplo de segundo orden

Consideremos:

$$G(s) = \frac{25}{s^2 + 4s + 25}$$

Comparando con la forma canónica:

$$\omega_n = 5\ rad/s$$

y:

$$2\zeta\omega_n = 4$$

por lo tanto:

$$\zeta = 0,4$$

La frecuencia natural amortiguada es:

$$\omega_d = \omega_n\sqrt{1 - \zeta^2}$$

y:

$$\omega_d \approx 4,58\ rad/s$$

---

26. Simulación de diferentes amortiguamientos con Python

```python
import matplotlib.pyplot as plt
from scipy import signal

wn = 5
zeta_values = [0.2, 0.7, 1.0, 2.0]

plt.figure()

for zeta in zeta_values:

    num = [wn**2]
    den = [1, 2*zeta*wn, wn**2]

    sistema = signal.TransferFunction(num, den)

    t, y = signal.step(sistema)

    plt.plot(t, y, label=f"zeta = {zeta}")

plt.grid(True)
plt.xlabel("Tiempo [s]")
plt.ylabel("Salida")
plt.title("Sistemas de segundo orden con distintos amortiguamientos")
plt.legend()
plt.show()
```

---

27. Comparación entre primer y segundo orden

Consideremos:

$$G_1(s) = \frac{1}{2s+1}$$

y:

$$G_2(s) = \frac{25}{s^2+4s+25}$$

PYTHON

```python
import matplotlib.pyplot as plt
from scipy import signal

G1 = signal.TransferFunction([1], [2, 1])
G2 = signal.TransferFunction([25], [1, 4, 25])

t1, y1 = signal.step(G1)
t2, y2 = signal.step(G2)

plt.figure()
plt.plot(t1, y1, label="Primer orden")
plt.plot(t2, y2, label="Segundo orden")
plt.grid(True)
plt.xlabel("Tiempo [s]")
plt.ylabel("Salida")
plt.title("Comparación de respuestas temporales")
plt.legend()
plt.show()
```

---

28. Comparación frecuencial

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

G1 = signal.TransferFunction([1], [2, 1])
G2 = signal.TransferFunction([25], [1, 4, 25])

w = np.logspace(-2, 2, 1000)

_, H1 = signal.freqresp(G1, w=w)
_, H2 = signal.freqresp(G2, w=w)

mag1 = 20*np.log10(np.abs(H1))
mag2 = 20*np.log10(np.abs(H2))

plt.figure()
plt.semilogx(w, mag1, label="Primer orden")
plt.semilogx(w, mag2, label="Segundo orden")
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Magnitud [dB]")
plt.title("Comparación de respuestas frecuenciales")
plt.legend()
plt.show()
```

---

29. Resonancia

Un sistema de segundo orden subamortiguado puede presentar una elevación de la magnitud alrededor de determinada frecuencia.

Este fenómeno se denomina **resonancia**.

La existencia y magnitud del pico resonante dependen especialmente del amortiguamiento.

Es un concepto importante en:

- sistemas mecánicos;
- sensores;
- actuadores;
- instrumentación biomédica;
- sistemas de medición;
- circuitos eléctricos.

---

30. Ejemplo biomédico integrador

Consideremos:

$$G(s) = \frac{100}{s^2 + 10s + 100}$$

Se obtiene:

$$\omega_n = 10\ rad/s$$

y:

$$\zeta = 0,5$$

El sistema es subamortiguado.

Podemos estudiar:

- respuesta al escalón;
- sobreimpulso;
- tiempo de establecimiento;
- frecuencia natural;
- respuesta en frecuencia;
- magnitud;
- fase.

Python

```python
import matplotlib.pyplot as plt
from scipy import signal

G = signal.TransferFunction([100], [1, 10, 100])

t, y = signal.step(G)

plt.figure()
plt.plot(t, y)
plt.grid(True)
plt.xlabel("Tiempo [s]")
plt.ylabel("Salida")
plt.title("Respuesta al escalón")
plt.show()

w, mag, phase = signal.bode(G)

plt.figure()
plt.semilogx(w, mag)
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Magnitud [dB]")
plt.title("Respuesta en frecuencia - Magnitud")
plt.show()

plt.figure()
plt.semilogx(w, phase)
plt.grid(True, which="both")
plt.xlabel("Frecuencia angular [rad/s]")
plt.ylabel("Fase [grados]")
plt.title("Respuesta en frecuencia - Fase")
plt.show()
```

---

31. Teorema del valor inicial

Bajo las condiciones correspondientes:

$$y(0^+) = \lim_{s \to \infty} sY(s)$$

Permite obtener el valor inicial sin calcular explícitamente la transformada inversa.

---

32. Teorema del valor final

Bajo las condiciones de estabilidad necesarias:

$$y(\infty) = \lim_{s \to 0} sY(s)$$

Estos teoremas son útiles para comprobar resultados obtenidos mediante simulación.

---

33. Una misma función, tres preguntas

Dominio temporal

¿Cómo responde el sistema después de aplicar una entrada?

Se estudian velocidad, oscilaciones, sobreimpulso, tiempo de establecimiento y valor final.

Dominio frecuencial

¿Cómo responde frente a diferentes frecuencias?

Se estudian ganancia, atenuación, fase, frecuencia de corte, ancho de banda y resonancia.

Polos y ceros

¿Qué características dinámicas pueden anticiparse a partir de $ H(s)$?

Se estudian estabilidad, modos dinámicos, rapidez y comportamiento oscilatorio.

---

34. Actividad de simulación: análisis completo

Analizar:

$$G(s) = \frac{36}{s^2 + 6s + 36}$$

Actividades

1. Identificar ω_n .
2. Determinar ζ .
3. Clasificar el sistema.
4. Obtener los polos.
5. Simular la respuesta al escalón.
6. Determinar aproximadamente el sobreimpulso.
7. Determinar el tiempo de establecimiento.

8. Obtener la respuesta en frecuencia.
9. Graficar magnitud en dB.
10. Graficar fase.
11. Identificar posibles efectos de resonancia.
12. Comparar los resultados temporales y frecuenciales.

---

35. Ejercicio aplicado a señales biomédicas

Supongamos:

$$x(t) = \sin(2\pi 2t) + 0,5\sin(2\pi 20t)$$

y:

$$G(s) = \frac{1}{0,05s + 1}$$

Preguntas

1. ¿Cuál es la frecuencia de corte?
2. ¿Qué componente debería atenuarse más?
3. ¿Qué ocurre con la amplitud de cada componente?
4. ¿Existe diferencia de fase?
5. ¿Cómo cambia la señal temporal de salida?

---

36. Simulación de una señal sinusoidal

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

G = signal.TransferFunction([1], [0.05, 1])

fs = 1000
t = np.arange(0, 10, 1/fs)

f = 2
x = np.sin(2*np.pi*f*t)

t, y, _ = signal.lsim(G, U=x, T=t)

plt.figure()
plt.plot(t, x, label="Entrada")
plt.plot(t, y, label="Salida")
plt.grid(True)
plt.xlabel("Tiempo [s]")
plt.ylabel("Amplitud")
plt.title("Respuesta a una señal senoidal")
plt.legend()
plt.show()
```

---

37. Analizando varias frecuencias

Una experiencia interesante consiste en repetir la simulación para distintas frecuencias:

```python
import numpy as np
from scipy import signal

frecuencias = [0.5, 2, 5, 10, 20, 50]

for f in frecuencias:

    x = np.sin(2*np.pi*f*t)

    _, y, _ = signal.lsim(G, U=x, T=t)

    print(f"Frecuencia:", f, "Hz")
```

El estudiante puede observar que el sistema responde de manera diferente según la frecuencia de la entrada.

Esto permite comprender experimentalmente el concepto de **respuesta en frecuencia**.

---

38. Relación entre dominio temporal y frecuencial

Existe una relación directa entre ambos análisis.

Un sistema que responde lentamente en el tiempo suele presentar una respuesta limitada para frecuencias altas.

En un sistema de primer orden:

$$G(s) = \frac{K}{\tau s + 1}$$

una constante de tiempo grande implica:

- respuesta temporal lenta;
- menor frecuencia de corte;
- menor capacidad para seguir variaciones rápidas.

Una constante de tiempo pequeña implica:

- respuesta temporal rápida;
- mayor frecuencia de corte;
- mayor ancho de banda.

---

39. Errores frecuentes

Confundir frecuencia $ f$ con frecuencia angular $ \omega$

$$\omega = 2\pi f$$

donde $ f$ está en Hz y $ \omega$ en rad/s.

Confundir ganancia con magnitud en dB

Si:

$$|H| = 2$$

entonces:

$$20 \log_{10}(2) \approx 6,02 \; dB$$

Confundir magnitud y fase

Representan propiedades diferentes.

Confundir frecuencia natural con frecuencia de corte

No son conceptos equivalentes.

Suponer que todo sistema de segundo orden presenta resonancia

La resonancia depende de los parámetros del sistema, especialmente del amortiguamiento.

Usar el teorema del valor final sin verificar estabilidad

El teorema tiene condiciones de aplicación.

---

40. Python como herramienta de simulación

La estructura básica utilizada en este capítulo es:

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
```

NumPy

Para:

- vectores de tiempo;
- frecuencias;
- operaciones matemáticas;
- generación de señales.

SciPy

Para:

- representar sistemas dinámicos;
- calcular respuestas temporales;
- calcular respuestas frecuenciales;
- simular entradas arbitrarias.

Matplotlib

Para:

- graficar señales;

- comparar sistemas;
- representar respuestas temporales;
- representar respuestas frecuenciales.

---

41. Ejemplo de análisis automatizado

PYTHON

```python
import numpy as np
from scipy import signal

G = signal.TransferFunction([25], [1, 4, 25])

polos = np.roots([1, 4, 25])

print("Polos:")
print(polos)

w, mag, phase = signal.bode(G)

print("\nMáxima magnitud [dB]:")
print(np.max(mag))
```

Python permite combinar cálculo numérico, simulación y visualización en un mismo entorno.
